MODULE DESIGN · CENG 396

STRATA

Module 2: Inertial Sensing & Data

Proposed software design: from raw IMU samples to calibrated, timestamped records

Instructor: Erkan Zergeroğlu

We build safety, not assumptions.

What the module does

This module takes raw acceleration and angular-rate samples from the IMUs, and produces a timestamped, calibrated acceleration record with health flags, plus a replayable recording.

This gives the project a measurement that is independent of the camera, and that can be cross-checked with the camera's data.

In one line

- Raw IMU samples
- Calibrated, timestamped X acceleration
- Health flag on every record
- Replayable log

Design status. Function names and interfaces in this document are proposed, not claims of completed implementation. The module can be developed and tested on its own, using a test board and recorded logs, without live inputs from the other modules.

Inputs and outputs

INPUTS

1. Raw acceleration samples
2. Device settings
3. Time reference
4. Angular-rate samples

MODULE 2

Inertial Sensing & Data

Read and timestamp

Check validity

Convert and calibrate

Filter and package

Save for replay

OUTPUTS

1. Calibrated acceleration record
2. Sensor health flags
3. Replayable recordings

Diagram to be refined. Outputs go to Module 4 and to a local log file.

The output record

1 Calibrated acceleration record

One record per sample, with these fields:

- **a** Acceleration along X
  in m/s², signed: positive and negative X directions are defined
- **b** Sensor source
  e.g. base IMU, floor 1, floor 2, floor 3
- **c** Timestamp
  when the sample was taken
- **d** Health flag
  status attached to the record

2 Sensor health flags

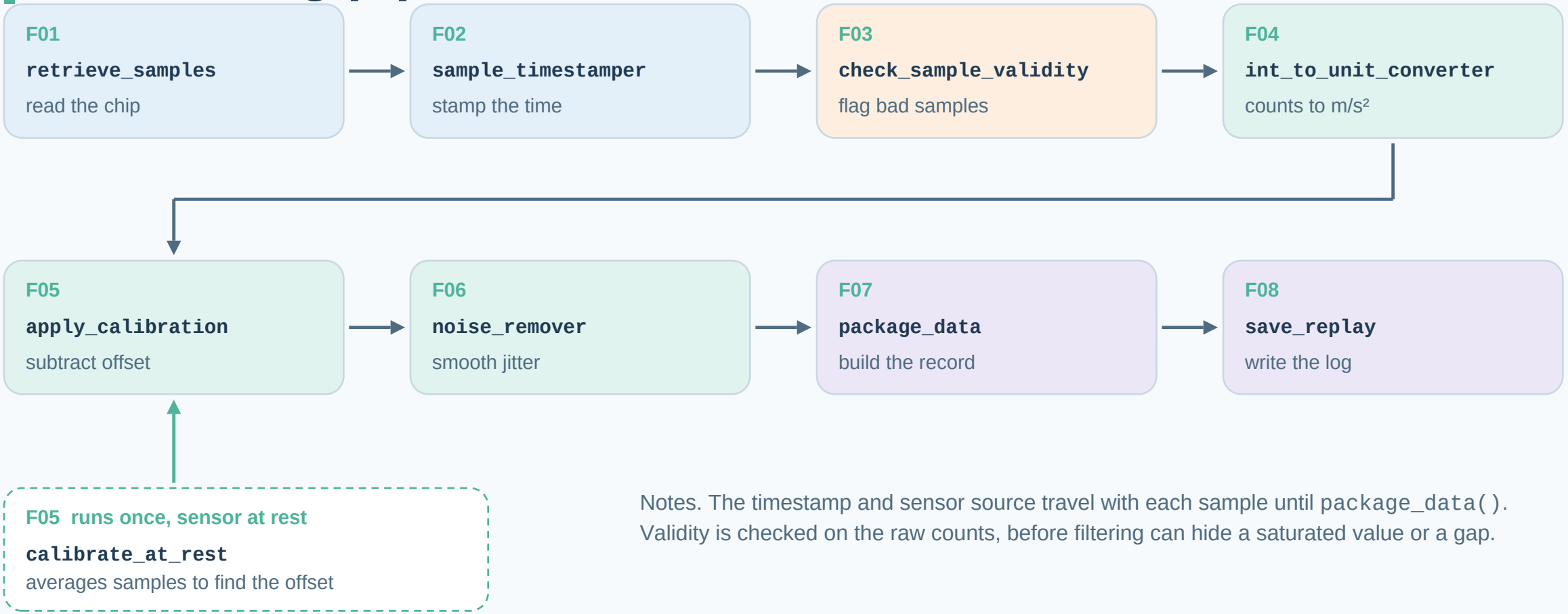
A status attached to each record:

- `OK`
- `saturated`
- `dropped sample`
- `sensor not responding`

3 Replayable recordings

Log files containing the same records (ideally with the raw samples too), which can be played back as if they were live data.

Processing pipeline



Notes. The timestamp and sensor source travel with each sample until `package_data()`. Validity is checked on the raw counts, before filtering can hide a saturated value or a gap.

Functions: acquire and check

Function	Inputs	Output	What it does
F01 `retrieve_samples`	sensor registers, device settings	raw integer acceleration and angular-rate counts, sensor source	Reads data off the chip.
F02 `sample_timestamper`	raw counts, time reference	raw sample + timestamp	Timestamps each sample so we know when it was taken.

The timestamp and sensor source travel with each sample until package_data().

Functions: validity check

Function	Inputs	Output	What it does
F03 `check_sample_validity`	raw counts, timestamp, previous timestamp, sensor range limits, sample period, previous raw values	health flag	saturated if the raw value sits at the range limit dropped sample if the timestamp gap is bigger than the sample period sensor not responding if the chip didn't answer or the value never changes OK otherwise

The health flag values match output 2. The flag is attached to every record by package_data().

Functions: units and calibration

Function	Inputs	Output	What it does
F04 `int_to_unit_converter`	raw counts, sensitivity for the chosen range	acceleration in m/s²	Turns the integer samples into m/s².
F05 `calibrate_at_rest`	many samples taken with the sensor still	offset (bias) value	Averages the samples to find the sensor's offset and any gravity leakage. Runs once, not on every sample.
F06 `apply_calibration`	converted sample, offset	corrected X acceleration	Subtracts the offset from every sample.

Functions: filter, package and save

Function	Inputs	Output	What it does
F07 **`noise_remover`**	corrected acceleration (last N samples)	filtered acceleration	Moving-average low-pass filter that smooths jitter. N is chosen after testing on real shaking.
F08 **`package_data`**	filtered X acceleration, timestamp, health flag, sensor source	one record in the shared contract format	Puts the fields together so every record looks the same.
F09 **`save_replay`**	records (and the raw samples)	log file that can be played back like live data	Writes everything to a file for offline filter tuning and for Module 4 development.